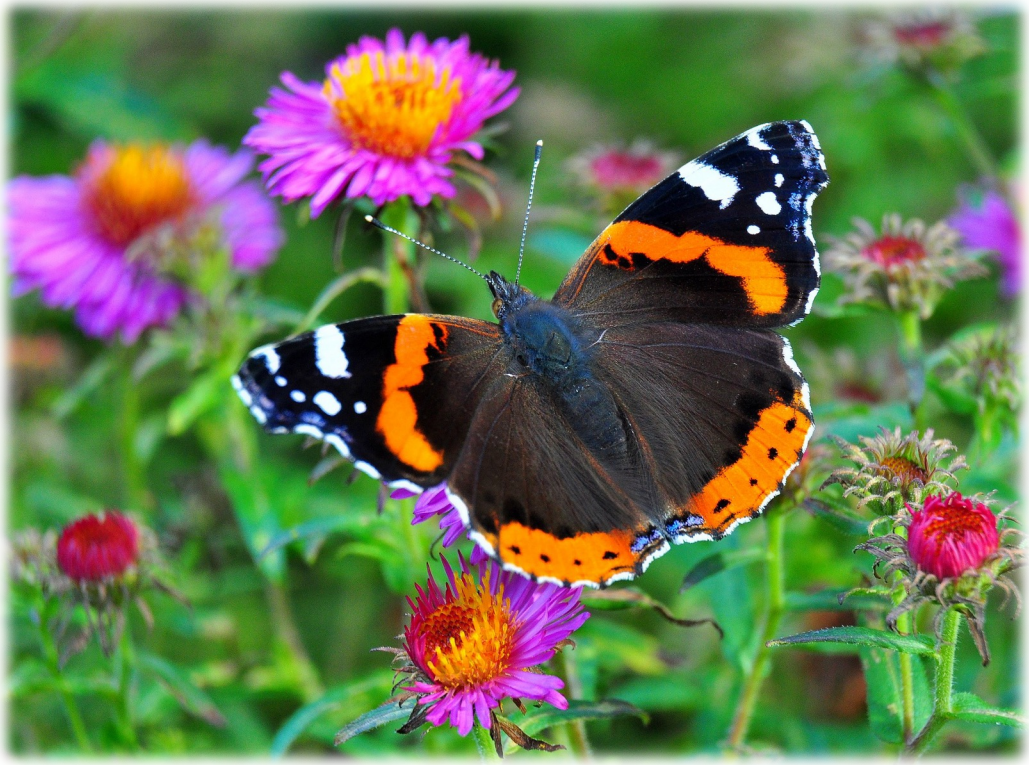


Photo by huems04 on pixabay

Lecture 11

–

Partitioning Clustering

Admiral (*Vanessa atalanta*)

- Ein sehr bekannter und weit verbreiteter Schmetterling der nördlichen Hemisphäre aus der Familie der Edelfalter (Nymphalidae). Der Name leitet sich von Atalanta, einer Jägerin aus der griechischen Mythologie ab.

- Er ist ein Wanderfalter, der im Frühjahr aus dem Mittelmeerraum über die Alpen nach Deutschland fliegt. Er erreicht eine Flügelspannweite von 55-65mm.

- Er hat samtig schwarze Vorderflügeloberseiten, auf denen etwa in der Mitte eine breite gezackte ziegelrote Binde verläuft. Die Weibchen haben in dieser fast immer einen kleinen weißen Fleck. Die tiefschwarzen Spitzen der Vorderflügel tragen einen großen weißen Balken und mehrere kleine weiße Flecken. Die Hinterflügel sind ebenfalls tief schwarzbraun gefärbt und tragen eine breite orangerote Binde am Flügelaußenrand. In dieser verläuft in der Mitte eine schwarze Punktreihe und im Hinterwinkel ein länglicher blauer Fleck. Am äußersten Rand aller vier Flügel verläuft eine sehr dünne weiße Linie, die kurz durch schwarze Punkte unterbrochen wird.

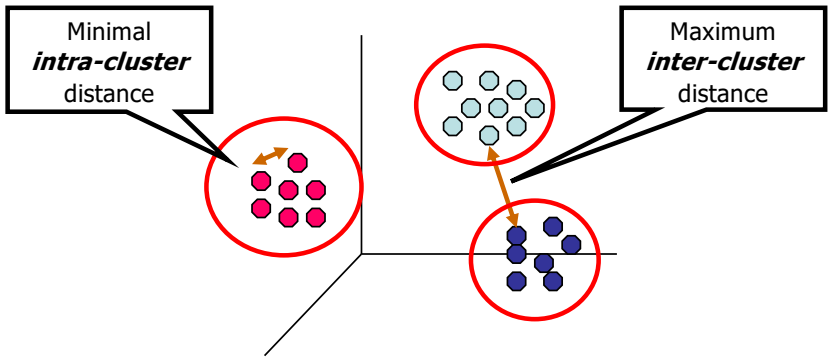
- Die Raupen fressen ausschließlich Brennnesseln.

Quellen:
* https://de.wikipedia.org/wiki/Admiral_(Schmetterling)
* https://www.infranken.de/ratgeber/tiere/10-der-schoensten-schmetterlinge-in-deutschland-praechtig-und-bedroht-art-5250577

Partitioning Clustering

Clusters and Clustering

- A *cluster* is a set of sample points
- Ideal case scenario:
  - Samples within a cluster are similar (regarding a metric)
  - Samples from different clusters should be very dissimilar



- Clustering (or: *cluster analysis*)
  - Process of identifying a finite number of clusters (categories groups) from a set of observations
  - Unsupervised learning: No labels given

“Cluster“ and clustering are to some extent ill-defined!

How many clusters?

Easy: 2

Or maybe 4?

…or even 6?

Types of Clustering Algorithms

- Structure of the clustering result
  - Partitioning: divide dataset into set of clusters in a single-level partition (flat result)
  - Hierarchical: build a multi-level structure of clusters (nested result)
- Membership type of objects
  - Exclusive: each object belongs to exactly one cluster
  - Non-Exclusive: object can belong to multiple clusters
- Membership rules
  - Hard: objects gets definite assignment(s)
  - Soft: object gets partial memberships, weights or probabilities
- Comments
  - Exclusive clustering is always hard, Non-Exclusive may be hard or soft
  - Soft can be converted to Hard: assign each object to the cluster with highest weight or probability

Examples of Clustering Algorithms

- k-means, k-medoids
  - Assign objects to clusters based on distance
  - partitioning, exclusive, hard
- DBSCAN
  - Assign objects to clusters based on density
  - partitioning, exclusive, hard
- GMM/EM (Gaussian Mixture Models optimized using the Expectation Maximization Alg.)
  - Use probability distribution
  - partitioning, non-exclusive, soft
- Agglomerative hierarchical clustering
  - hierarchical
  - Before cut: hard. After cut: exclusive, hard
  - Can be considered exclusive or non-exclusive before cut

Example 1: Clustering for Customer Segmentation in e-commerce or retail

- Goal: Group similar customers without predefined labels
- Typical use cases
  - Personalization
  - Campaign design
  - CRM
  - Recommendation support
- Core Idea:
  Customers with similar behavioral patterns are placed in the same cluster

Possible features

- purchase frequency
- average basket value
- return rate
- discount sensitivity
- preferred product categories

Example 2: Clustering Documents by Topic

- Goal: Group similar documents based on their content
- Typical use cases
  - Topic discovery
  - News organization
  - Support ticket triage
  - Corpus exploration

Possible representations
- TF-IDF vectors
- sentence embeddings
- document embeddings
- keyword frequencies

- Core Idea:
  Text is first transformed into a numerical representation and then clustered by similarity

Example 3: Clustering System Behavior and Network Activity

- Goal: Discover typical behavior patterns in systems, devices, or network sessions

- Typical use cases
  - Usage profiling
  - Monitoring
  - Exploratory cybersecurity analysis
  - Operations analytics

Possible features
- requests per minute
- average packet size
- login frequency
- CPU usage
- session duration
- error rates

- Core Idea:
  Clustering helps separate different patterns of normal and unusual behavior

Example 4: Clustering Patients or Regions with Similar Profiles

- Goal: Identify groups with similar profiles in healthcare or public-sector data

- Typical use cases
  - Population analysis
  - Planning and resource allocation
  - Risk stratification
  - Policy support

Possible features
- age
- BMI
- blood pressure
- income indicators
- age structure of a region
- healthcare access

- Core Idea:
  Clustering can reveal structure in multivariate data where no labels exist

Example 5: Clustering for Data Reduction and Prototypes

- Goal: Summarize many similar observations by a smaller set of representative prototypes
- Typical use cases
  - Faster training
  - Reduced storage
  - Simpler exploratory models
  - Preprocessing for nearest-neighbor methods

Downstream use
- prototype-based classification
- compressed training sets
- faster similarity search
- reduced memory requirements

- Core Idea:
  Clustering can replace many redundant data points with centroids, medoids, or representative instances

Example 6: Clustering for feature engineering

- Goal: Use clustering to generate new features for a later classification or regression model
- Typical use cases
  - Churn prediction
  - Fraud detection
  - Customer analytics
  - Predictive maintenance

Cluster-derived features

- cluster ID
- one-hot encoded cluster membership
- distance to centroid 1
- distance to centroid 2
- distance to centroid …

- Core Idea:
  Cluster membership or distance to cluster centers can be added as new features

Example 7: Cluster first, then train specialized models

- Goal: Identify distinct subpopulations and
  train one model per subgroup instead of one global model

- Typical use cases
  - Predictive maintenance for different machine profiles
  - Demand forecasting for different store types
  - Healthcare risk models for different patient groups

Possible workflow
1. cluster the data
2. identify subgroups
3. train one model per cluster
4. compare with one global model

- Core Idea:
  When the population is heterogeneous, one global model may be too coarse

Example Summary

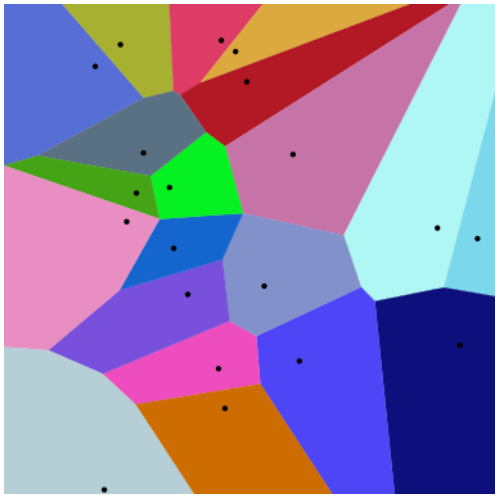
- After clustering, we interpret the groups → clustering is the main task
  - E.g., customer segmentation, document clustering, …
- After clustering, we use the groups to help another model → clustering is a pre-processing step
  - data reduction before learning
  - feature construction for learning
  - data partitioning to train specialized models
- Clustering answers the same core question:
  Which objects are similar to each other, based on the chosen features?
- Follow-up questions
  - How do we find similar objects?
  - How do we represent the objects?
  - How do we measure similarity?

Partitioning Clustering

1. Clustering
2. Partitioning Basics
3. k-means Clustering
4. Feature Spaces and Distance Measures
5. k-medoids Clustering (PAM)

Voronoi Diagram

- Partitioning of Euclidean plane
- Defined by Voronoi cells (points) and distance metric



Euclidean distance

$$\ell_2 = d\left[(a_1, a_2), (b_1, b_2)\right] = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}$$

Manhattan distance

$$d\left[(a_1, a_2), (b_1, b_2)\right] = |a_1 - b_1| + |a_2 - b_2|$$

Source: wikipedia.de

Partitioning Basics

- Goal: Find partition in *k* clusters with minimal cost

- Local optimization
  - Chose *k* clusters (at random)
  - Assign data to clusters based on current estimate
  - Re-estimate cluster based on assignments

- Types of cluster representatives (centroids)
  - Mean value (k-means)
  - “Center-most“ element in cluster (k-medoid/PAM)
  - Gaussian distribution (via EM, next chapter)

Partitioning Clustering

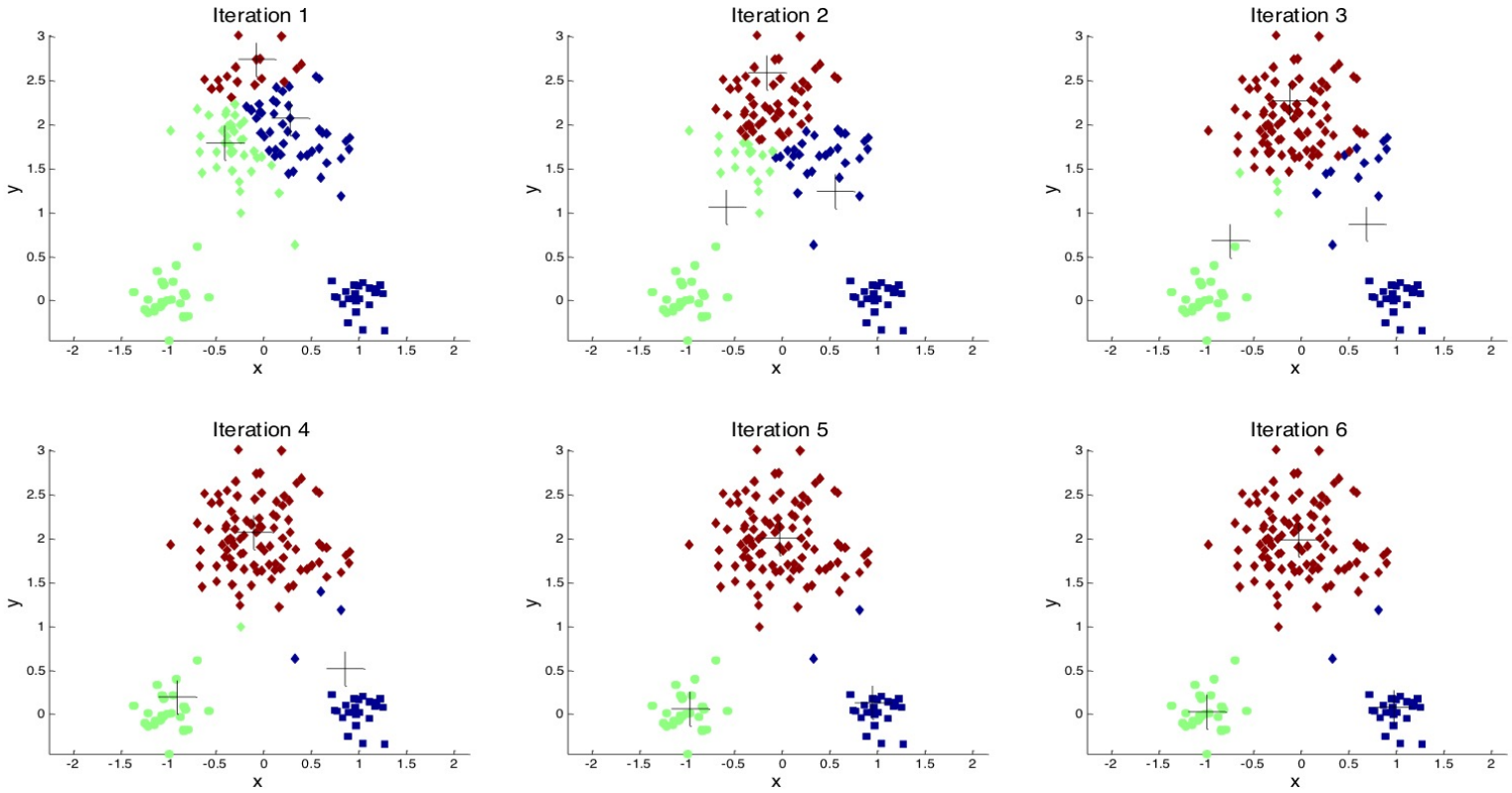
1. Clustering
2. Partitioning Basics
3. k-means Clustering
4. Feature Spaces and Distance Measures
5. k-medoids Clustering (PAM)

k-means

- Data: X = (x^p_1, ..., x^p_d) in Euclidean space
- (Typically) uses Euclidean distance (L_2-norm)
- Centroid element is mean value of current assignments
- *k* is given apriori (must be chosen)

k-means(data, dist, k)
1: select *k* values as initial centroids c(k)
2: REPEAT
3: Assign each data point to cluster based on distance to centroid
4: Recompute centroids
5: UNTIL change in centroids less than ε

k-means: Example 1

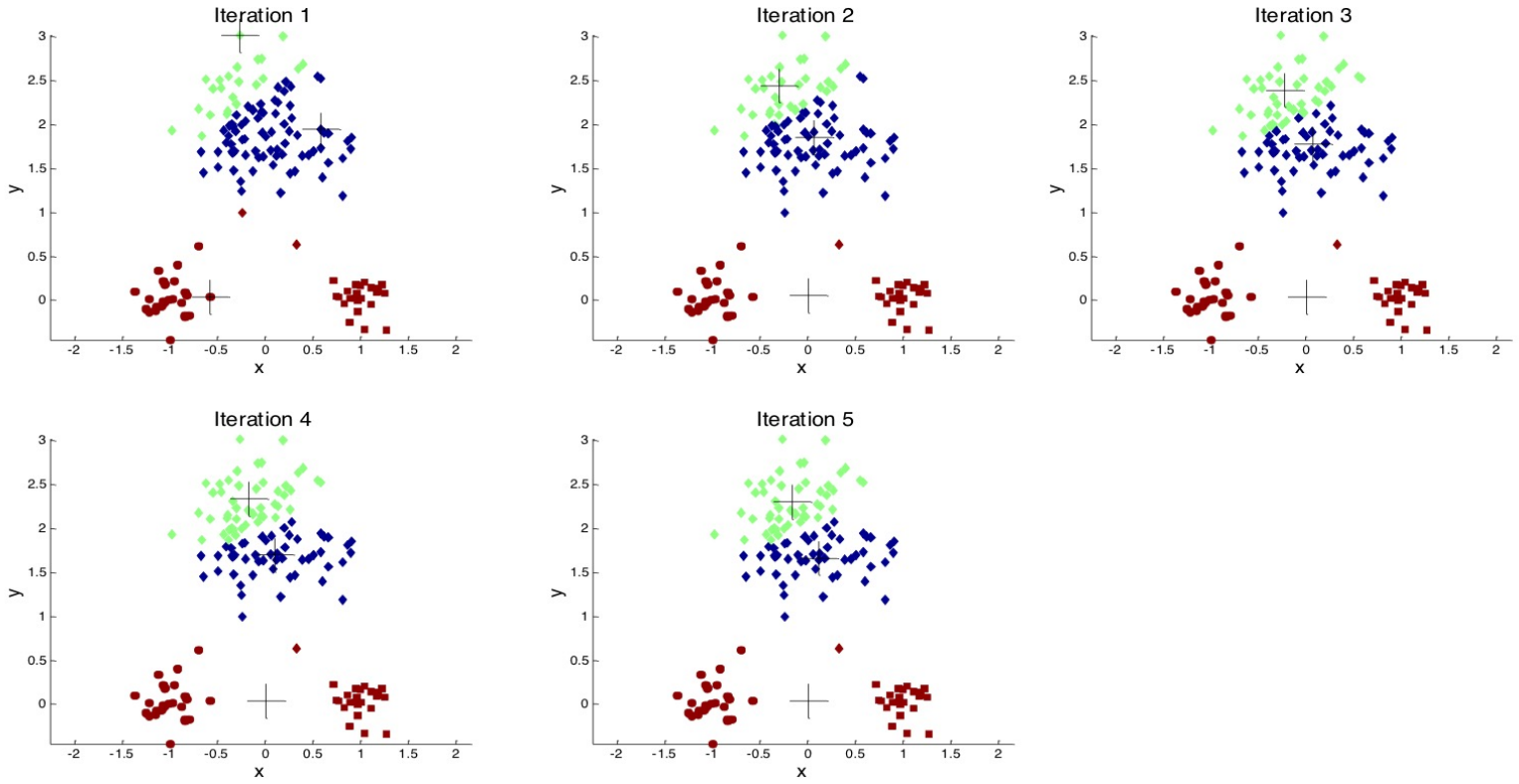


k-means: Discussion

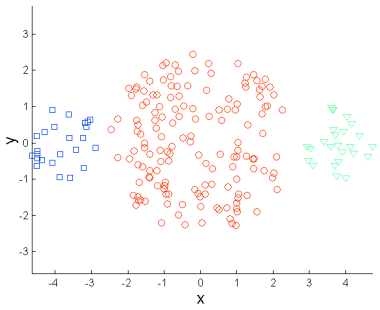
- Pros
  - (Fairly) efficient: O(nkt) with n samples, k clusters, t iterations
  - Straight-forward implementation

- Cons
  - Does not work well on non-numeric data (curse of dimensionality)
  - k must be specified
  - Prone to noise and outliers
  - Clusters should be convex (in terms of Euclidean geometry)
  - Often converges in local optimum – initial centroids critical!

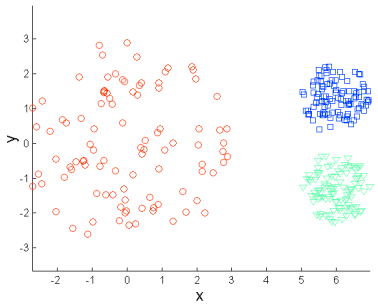
k-means: Initial Centroids



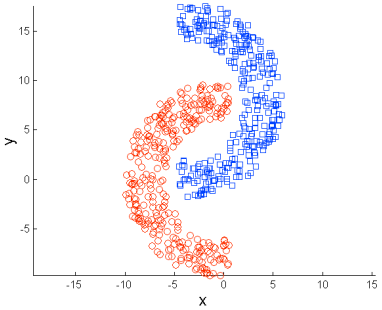
k-means: Non-Convex Clusters



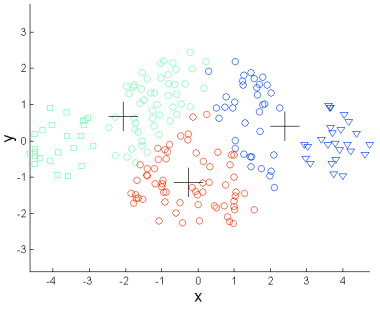
Data of different size



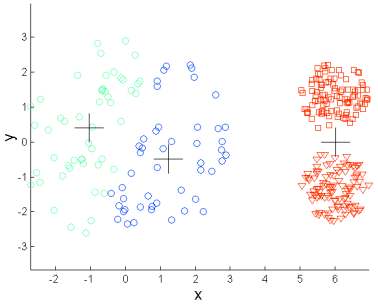
Data of different density



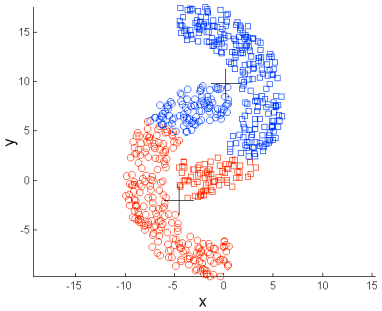
non-convex clusters



k-means (3 Clusters)



k-means (3 Clusters)



k-means (2 Clusters)

k-means: Issues

- Non-numeric data
  - ➔ k-medoids (later today)
- Choice of *k*
  - ➔ over-allocate and reduce
- Noisy data or outliers
  - ➔ k-medoids
- Non-convex data
  - ➔ over-allocate *k*
- Convergence in local optimum
  - ➔ heuristics on re-allocation
- Risk of initial centroids
  - ➔ re-run multiple times with different seeds (see cross-validation)

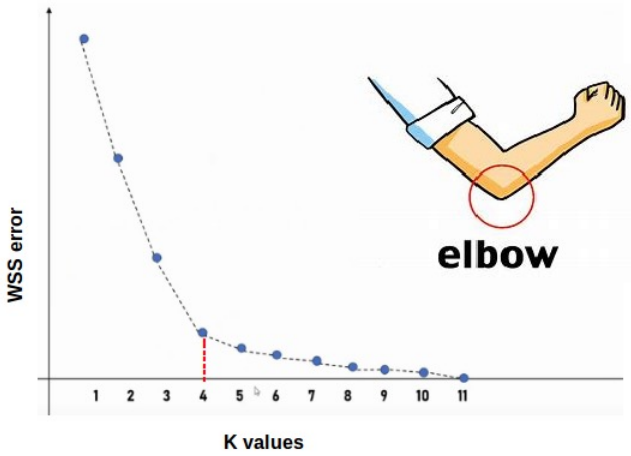
Exercises

Exercise 1

k-means

Choosing k: The Elbow Method

- Idea
  - Look for the point where adding more clusters yields only small additional improvement: Run k-means for several values of k and compute the within-cluster sum of squares (WCSS) (also called inertia)
    - as k increases, WCSS always decreases
    - more clusters fit the data more closely
    - but after some point, the improvement becomes much smaller
- Elbow principle: Choose k near the “elbow” of the curve:
  - before the elbow: large gains
  - after the elbow: diminishing returns
- Interpretation: elbow suggests a reasonable trade-off between
  - model simplicity
  - cluster compactness



Source: https://medium.com/@gowthamiarva28/exploring-clustering-techniques-k-means-k-means-hierarchical-clustering-and-the-elbow-23eda62d233a

Choosing k: The Silhouette Coefficient

- Idea: measure how well points fit their own cluster compared to next best alternative cluster
- Silhouette Coefficient: For one point i :

  $a(i)$: average distance to points in the same cluster

  $b(i)$: average distance to points in the nearest other cluster

$$s(i) = \frac{b(i) - a(i)}{\max(a(i),\ b(i))}$$

- Interpretation
  - $s(i) \approx 1$: well assigned
  - $s(i) \approx 0$: near a boundary
  - $s(i) < 0$: possibly misassigned
- For choosing k:
  - compute the average silhouette score for several values of k
  - prefer solutions with higher average silhouette values
  - combine this with interpretability and domain knowledge

Partitioning Clustering

1. Clustering
2. Partitioning Basics
3. k-means Clustering
4. Feature Spaces and Distance Measures
5. k-medoids Clustering (PAM)

Why clustering depends on representation

- Clustering groups representations of objects in a feature space, not real-world objects directly
- Before clustering, real-world objects must be represented by features (in a feature space)
- Important implications
  - similarity is computed in the representation (feature space)
  - different representations can lead to different clusterings
  - clustering is exploratory, but not assumption-free
- Consider:

  If we cluster the same customers
  - once by demographics and
  - once by buying behaviour,

  should we expect the same groups?

From objects to feature spaces

- Each object becomes a vector, and similarity is defined in the resulting feature space
- Feature space: the space induced by the chosen feature
- Basic Idea
  - each feature corresponds to a dimension
  - each observation becomes a vector
  - neighbourhood and similarity are defined in this space
- The algorithm sees numerical representations, not semantic meaning!
- Consider:
  - What changes when we add one more feature to the dataset?
  - Why is a feature space more than just “a table with columns”?

The same objects in different feature spaces

- Closeness depends on the chosen coordinates, so the same objects can produce different valid clusterings

- Example
  - the same customers described by different features
    - age, income
    - purchase frequency, basket value
    - return rate, product diversity
  - Possible consequence
    - different neighbourhoods
    - different cluster structures
    - different interpretations

- Consider:
  - Which feature space would be more useful for marketing: age and income, or purchase behaviour? Why?
  - Can two different clusterings of the same objects both be valid?

Feature choice determines detectable structure

- The selected features determine
  which similarities become visible & which structures remain hidden

- Feature choice matters
  - relevant features can reveal meaningful structure
  - Irrelevant/noise features can blur cluster structure/hide useful patterns
  - redundant features can overweight one aspect
  - engineered features can make structure more visible

- Examples
  - customer clustering: demographics vs. behaviour
  - text clustering: TF-IDF vs. embeddings
  - images: raw pixels vs. learned features

- Consider:
  - Which is likely to be more useful for document clustering: raw word counts or embeddings? Why?

Why scale matters: Scaling/Normalization is a modelling decision

- In distance-based clustering, variables with larger numeric ranges can dominate similarity
- Scaling can improve comparability, but it also changes the implicit weighting of features
- Implication / Interpretation
  - Euclidean distance is scale-sensitive
    - If scales differ strongly, the distance reflects the large-scale variables much more than the smaller-scale variables.
  - Scaling can give features more comparable influence
  - Normalization changes geometry differently
  - Scaling/Normalization may be useful, but are not automatically “correct”
  - Scaling/Normalization change which dimensions contribute most strongly to closeness
- Consider:
  - Does Scaling/Normalization remove assumptions, or introduce new ones?
  - Can there be situations where not scaling is actually the better choice?

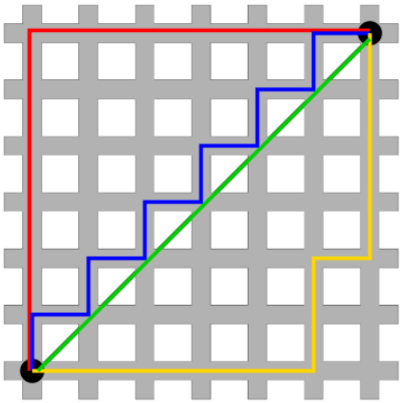
Example:

Two customers differ by:
- age: 8 years
- income: 25,000 euros

Without scaling:
Income dominates
in the Euclidean distance

Euclidean and Manhattan Distance

- Euclidean distance measures straight-line closeness
  - intuitive geometric interpretation but sensitive to scale
  - aligns naturally with k-means
  - favours compact, roughly spherical cluster structures
  - Use when
    - features are numeric
    - geometric closeness is meaningful
    - centroid-based clustering is intended



- Manhattan distance measures coordinate-wise deviation
  - “city block” distance
  - Use when
    - coordinate-wise differences matter
    - additive deviations are more meaningful than straight-line geometry

- Consider:
  - What changes when we switch from Euclidean to Manhattan distance?

Cosine Similarity: Direction Instead of Magnitude

- Cosine similarity compares vector orientation, making it useful when direction matters more than size
  - insensitive to overall magnitude
  - emphasizes relative composition
  - useful when “same direction” matters more than “same size”
  - Use for
    - text vectors
    - embeddings
    - user or item profiles
    - high-dimensional sparse data
- Key contrast
  - Euclidean/Manhattan ask: how far apart are the vectors?
  - Cosine asks: how aligned are the vectors?
- Consider:
  - Why can a long document and a short document still be very similar under cosine similarity?

Binary, Categorical and Ordinal Data

- Binary Data: Hamming distance: Count number of times where the values differ
  - Consider two binary vectors (of same dimension) as a whole; equal weight to individual differences
  - Example Hamming distances:
    - "karolin" and "kathrin" is 3
    - "karolin" and "kerstin" is 3
    - **1011101** and **1001001** is 2
    - **2173896** and **2233796** is 3

- Categorical Data: Count Agreement: Count agreement *m* in a total of *p* attributes
  - Categorial data is a generalization of binary data: consider all categorial fields at the same time
  - Alternatively: convert/expand to binary vectors (e.g. in python with the function *get_dummies*)

$$d(i,j) = \frac{p-m}{p}$$

- Ordinal Data: Treat as Categorical or as Numeric

- Consider:
  - Why is coding device type as smartphone = 1, tablet = 2, laptop = 3 potentially misleading for Euclidean distance?
  - Why is an ordinal scale such as low–medium–high not already fully numeric?

Representation, distance, and data type must match

- A useful similarity notion depends on what the feature values mean, not only on how they are stored
  - Numeric data
    - magnitude differences often meaningful
    - Euclidean or Manhattan can be natural starting points
  - Binary data
    - yes/no, present/absent, state indicators
    - 0/1 encoding is numeric in storage, but not always quantitative in meaning
  - Categorical data
    - nominal categories have no natural order
    - arbitrary integer coding can create artificial distances
  - Ordinal data
    - order exists
    - spacing between levels may not be equal
  - Mixed data
    - many real datasets combine several data types
    - one single Euclidean treatment is often insufficient

Common misconceptions

- Clustering behavior is determined jointly by representation, scaling, distance, and data type
- Common misconceptions
  - “The algorithm finds the true clusters.”
  - “More features are always better.”
  - “Scaling is just technical housekeeping.”
  - “Distance is an objective property of the data.”
  - “Everything can be encoded numerically, so Euclidean distance will work.”
  - “0/1 variables are just ordinary numeric variables.”
  - “Ordinal data is the same as interval-scale numeric data.”
  - “k-means works for any table-shaped dataset.”
- Key takeaways
  - representation defines similarity
  - feature choice determines detectable structure
  - scaling/normalization changes geometry
  - distance is a modelling choice
  - data type matters

Exercises

Exercise 2

Distance Metrics

Partitioning Clustering

1. Clustering
2. Partitioning Basics
3. k-means Clustering
4. Feature Spaces and Distance Measures
5. k-medoids Clustering (PAM)

What k-Means Assumes

- k-means is a specific clustering method for data where Euclidean geometry and means are meaningful

  k-means is most natural when
  - features are numeric
  - Euclidean geometry is meaningful
  - means are interpretable cluster centers
  - scale has been handled appropriately
  - compact centroid-based clusters are plausible

- From k-means to k-medoids (PAM algorithm)
  - k-medoids requires a distance metric, but not an Euclidean space

k-medoids / PAM Algorithm

- PAM = Partitioning around Medoid (selected from data)
  - Medoid: the feature vector of a cluster, whose cumulative dissimilarity to all other features is minimal (not necessarily unique)
- Only requires a distance metric, but not Euclidean space
- Quality/cost: cumulative distance to medoids h: $TD(h) = \sum_{c \in Cluster(h)} d(c,h)$
- Measure improvement in terms of distance to new medoid i: $TC(i,h) = \left(\sum_{c \in Cluster(h)} d(c,i)\right) - TD(h)$

PAM(data, dist, k)
1: Chose *k* samples as medoids
2: assign data based on distance to medoid
3: REPEAT
4: For each pair of sample i and medoid h, compute TD(i,h)
5: For each pair data(i), data(h): if TC(i,h) < 0 # only points within cluster
6: replace h by i as medoid
7: update cluster assignment
8: UNTIL medoids constant

PAM: Discussion

- More robust w.r.t. noise/outliers, since medoid selected from data
- Complexity: $O(k(n-k)^2)$ for n samples and k clusters
  → ok for small data sets (pairwise distances!)
- Guaranteed global optimum: try all permutations
- Can cache pairwise distances for smallish data sets

Key Takeaways

- Clustering
  - → unsupervised learning
- k-means
  - Probably the most popular clustering algorithm
  - Only in Euclidian Vector Space
- Feature Spaces and Distance Measures
  - Central to achieving a sensible result - need to be carefully chosen
  - Depend on data type / semantics
- k-medoids (PAM)
  - For non-Euclidian data